Introdução

Python se tornou uma das linguagens centrais no ecossistema de dados. Entretanto, escrever scripts e manter um ambiente de desenvolvimento sustentável ao longo do tempo são duas coisas diferentes. À medida que projetos evoluem surgem desafios de empacotamento, gestão de dependências, integração contínua, reprodutibilidade e qualidade do código.

Ferramentas modernas de desenvolvimento Python ajudam a enfrentar esses desafios. Elas permitem:

- Gerenciar ambientes independentes e versões de Python.
- Instalar e atualizar dependências de forma controlada.
- Empacotar módulos e componentes reutilizáveis.
- Executar suites de testes e validações automatizadas.
- Automatizar fluxos operacionais repetíveis.
- Integrar com CI/CD para manter confiabilidade e rastreabilidade.

Essas ferramentas não são estritamente necessárias para escrever scripts isolados, mas tornam-se essenciais quando o código precisa ser executado todos os dias, em múltiplos ambientes, por múltiplos usuários, com expectativas de performance, qualidade e manutenção.

No contexto de dados, isso se manifesta em tarefas como:

- Controlar upgrades de dependências críticas (pandas, polars, dbt, pyspark, boto3, requests, airflow etc.).
- Publicar pacotes internos para compartilhamento entre equipes.
- Garantir que pipelines continuem funcionando apesar de mudanças no schema ou na plataforma.
- Automatizar testes de dados e validações antes de cargas.
- Reduzir custo operacional ao remover fricções do dia a dia.

Ferramental também traz complexidade, portanto deve ser adotado com propósito. O objetivo não é acrescentar ferramentas porque são novidade, mas facilitar a vida do desenvolvedor e tornar o ecossistema mais previsível, produtivo e seguro.

Este material ajuda profissionais de dados que já utilizam Python e querem evoluir para uma prática mais madura, reprodutível, testável, empacotável e operável, capaz de sustentar sistemas críticos de dados em produção, integrações com plataformas e longas janelas de manutenção.

O Ciclo de Release do Python

Python segue um ciclo de lançamentos anual, com uma nova versão publicada todo mês de outubro. Assim, é comum ver sequências como 3.12 em 2023, 3.13 em 2024 e 3.14 em 2025.

O esquema de versionamento **major.minor.micro** significa que um número como 3.11.5 expressa três níveis distintos de atualização: o primeiro (major) representa mudanças disruptivas e raras, o segundo (minor) traz novos recursos e melhorias relevantes para a linguagem e para o interpretador, e o terceiro (micro) cobre correções de bugs.

As releases minor: como 3.10, 3.11 e 3.12, são onde surgem novas funcionalidades, otimizações no interpretador, avanços em áreas como tipagem estática, async ou pattern matching, e melhorias gerais de desempenho. Importante notar que essas versões minor são projetadas para não quebrar código existente; a filosofia é evoluir sem causar rupturas no ecossistema.

Quando a linguagem precisa remover ou alterar algo de forma incompatível, primeiro marca o recurso como deprecated e só efetiva a mudança após um período de cerca de dois anos, permitindo que bibliotecas, projetos e sistemas adaptem-se com tranquilidade.

Versões major, como foi a transição do Python 2 para o Python 3, são reservadas para mudanças realmente estruturais e não seguem um ritmo frequente, até o início de 2024, por exemplo, um Python 4 nem está no radar. Esse modelo combina inovação com estabilidade operacional, o que é essencial para equipes e plataformas que executam Python em produção, especialmente em contextos de automação, engenharia de dados e machine learning.

Ambientes Virtuais

Trabalhar com Python por muito tempo inevitavelmente leva a um problema prático: diferentes projetos exigem diferentes versões de bibliotecas. Se todos os pacotes forem instalados diretamente no Python do sistema, eles passam a coexistir no mesmo espaço de nomes importáveis (namespace). Isso faz com que dependências de projetos distintos se misturem, dificultando testes, tornando a depuração mais confusa e prejudicando a reprodutibilidade dos resultados. Além disso, esse modelo impede que dois projetos utilizem versões conflitantes do mesmo pacote, já que apenas uma versão pode ocupar o ambiente global.

Esse problema se agrava porque, em muitas distribuições modernas de Linux, o Python do sistema é marcado como “externally managed”. Isso significa que ele é controlado pelo gerenciador de pacotes do sistema operacional e não deve ser modificado diretamente com pip. Na prática, isso bloqueia instalações globais e reforça ainda mais a necessidade de trabalhar com ambientes isolados.

É nesse contexto que surgem os ambientes virtuais. Um ambiente virtual é um ambiente leve e separado da instalação global do Python, que contém seus próprios pacotes de terceiros e funciona como uma camada isolada para cada projeto. A maior parte do ambiente, como a biblioteca padrão (stdlib) e o interpretador, é herdada da instalação global, mas os pacotes instalados dentro do ambiente virtual pertencem exclusivamente a ele e só podem ser importados pelo interpretador desse ambiente.

Internamente, um ambiente virtual não replica uma instalação completa do Python. Ele mantém apenas seus próprios pacotes de terceiros (aqueles instalados via pip) e reutiliza tanto o interpretador quanto a biblioteca padrão da instalação global que o criou. Por isso, a estrutura de diretórios de um venv se parece com a de uma instalação Python, mas sem duplicar todos os módulos da biblioteca padrão.

O executável python dentro do ambiente virtual funciona essencialmente como um apontador para o interpretador real do sistema:

- no Linux e no macOS, ele é um link simbólico;
- no Windows, é um pequeno wrapper que invoca o interpretador pai já configurado para usar os pacotes isolados do venv.

Cada ambiente virtual já inclui um pip próprio, pronto para instalar pacotes dentro dele. Assim, o fluxo básico torna-se: criar o ambiente, ativá-lo, instalar dependências e então executar seu código dentro desse contexto isolado.

Entretanto, quando você cria um ambiente virtual, ele não vem necessariamente com a versão mais recente das ferramentas de instalação do Python. Ele copia a mesma versão de pip e setuptools presente na instalação global que o criou. Por isso existe a opção:

--upgrade-deps

que permite criar o ambiente já atualizando essas ferramentas essenciais.

Também é possível criar um ambiente virtual sem incluir o instalador de pacotes. Isso faz sentido em cenários onde a instalação de dependências é controlada por outro gerenciador (como ferramentas de empacotamento, sistemas corporativos ou ambientes científicos), e o papel do ambiente virtual passa a ser apenas fornecer um interpretador isolado, não um gerenciador de pacotes.

Como ainda é comum que usuários instalem pacotes sem perceber se estão dentro ou fora de um ambiente virtual, algumas configurações foram criadas para evitar esse tipo de erro. Um exemplo é:

pip config set global.require-virtualenv

que impede instalações acidentais no ambiente global do sistema, protegendo a organização, a reprodutibilidade e a manutenção dos projetos.

Cenário 1: O fluxo tradicional: venv + pip

No fluxo mais básico e tradicional, o usuário cria um ambiente virtual com o módulo padrão do Python:

python -m venv .venv

Esse comando cria um diretório .venv contendo um interpretador isolado e sua própria pasta site-packages. Para que esse ambiente seja efetivamente usado, ele precisa ser ativado:

- No Windows: **.venv\Scripts\activate**
- No Linux/macOS: **source .venv/bin/activate**

Após a ativação, o terminal passa a utilizar o python e o pip daquele ambiente específico, e não mais os do sistema global. O prompt normalmente muda para indicar que o venv está ativo, reduzindo o risco de confusão. Para sair do ambiente, basta executar:

deactivate

Esse modelo funciona bem e é amplamente utilizado, mas possui algumas limitações práticas. **Ele depende fortemente do Python exato que foi usado para criar o ambiente e não faz uso avançado de cache ou mecanismos mais robustos de descoberta de interpretadores. Em máquinas com múltiplas versões de Python, isso pode levar à criação de ambientes com a versão errada sem que o usuário perceba.**

Como o Python “sabe” que está dentro de um venv?

O funcionamento dos ambientes virtuais é engenhoso. O Python não duplica sua instalação dentro do venv; o interpretador lá dentro é essencialmente o mesmo da instalação global. Para distinguir se está rodando dentro de um ambiente virtual, o Python observa de onde o comando python foi executado.

Se esse executável estiver dentro de um diretório que contém um arquivo chamado pyvenv.cfg, o interpretador entende que está dentro de um ambiente virtual. Esse arquivo funciona como um marcador do venv.

Ao detectar o pyvenv.cfg, o Python passa a procurar pacotes instalados via pip no site-packages do ambiente virtual, e não no sistema global. O próprio pyvenv.cfg contém um campo chamado home, que aponta para a instalação original do Python. É dessa instalação "pai" que o ambiente virtual herda a biblioteca padrão (stdlib), como os, sys, json, entre outros. Assim, o venv consegue herdar o que precisa do Python global, mas isolar completamente os pacotes de terceiros do projeto.

Existe ainda a opção:

--system-site-packages

que permite ao ambiente virtual enxergar pacotes instalados globalmente. Porém, isso não é recomendado, pois quebra o isolamento e recria o problema de misturar dependências de projetos diferentes.

Por fim, é importante notar que o pip não decide sozinho onde instalar pacotes: ele consulta o interpretador Python em execução para determinar o local correto. Por isso, usar:

py -m pip install pacote

garante que o pip está associado exatamente ao interpretador e ao ambiente virtual desejado. Já chamar apenas pip depende do que está no PATH do sistema e pode acabar usando o pip errado, instalando pacotes no ambiente errado, um erro comum quando há vários Pythons coexistindo.

Cenário 2: Um passo além: virtualenv + pip

À medida que projetos e equipes crescem, o fluxo simples venv + pip pode começar a mostrar limitações, especialmente em máquinas com múltiplas versões de Python, ambientes corporativos ou pipelines automatizados.

Nesse contexto surge o virtualenv, uma ferramenta mais "industrial" de criação de ambientes virtuais. Em vez de usar o módulo padrão venv, o usuário cria o ambiente com algo como:

virtualenv .venv

Embora o resultado final ainda seja um ambiente virtual funcional, o virtualenv é mais robusto na forma como o constrói. Enquanto o módulo padrão venv basicamente utiliza o Python que foi chamado. Por exemplo, quando alguém

executa **python -m venv .venv**, o ambiente será criado exatamente com aquele interpretador que estiver no PATH, mesmo que não seja a versão ideal para o projeto.

Já, o virtualenv adota uma estratégia mais inteligente, ele tenta descobrir de maneira mais confiável quais versões do Python estão disponíveis no sistema, o que é especialmente importante em máquinas onde coexistem múltiplas instalações, como Python do sistema. Assim, ao executar algo como **virtualenv -p python3.12 .venv**, o virtualenv procura ativamente pelo interpretador correto, verificando sua existência e **garantindo que o ambiente seja criado exatamente com a versão desejada**, reduzindo o risco de se trabalhar inadvertidamente com a versão errada do Python.

Além disso, o virtualenv utiliza mecanismos de caching interno para acelerar a criação de novos ambientes. Em vez de reconstruir completamente a estrutura do venv toda vez, como ocorre no fluxo mais simples com python -m venv .vec. **Ele pode reaproveitar componentes já preparados anteriormente quando identifica que um ambiente está sendo criado com a mesma versão de Python. Na prática, isso significa que, se um ambiente com Python 3.12 já foi criado antes, uma nova execução de virtualenv .venv tende a ser significativamente mais rápida, pois parte do trabalho já foi realizado e armazenado em cache**. Essa característica se torna particularmente relevante em contextos de CI/CD e containers, onde ambientes virtuais são frequentemente recriados do zero a cada build ou deploy.

Nesses cenários, o uso do virtualenv reduz tempo de execução, consumo de recursos e risco de falhas, oferecendo uma base mais previsível e estável para equipes que trabalham em ambientes distribuídos e automatizados, mesmo mantendo o uso tradicional do pip para instalação de pacotes dentro desse ambiente.

Se o Cenário 1 (venv + pip) representa a abordagem básica e o Cenário 2 (virtualenv + pip) representa uma versão mais robusta e profissional desse fluxo, o próximo passo lógico é perguntar:

- E se houvesse uma ferramenta que combinasse a robustez do virtualenv com um instalador de pacotes muito mais rápido e seguro por padrão?

É exatamente nesse ponto que entra o uv, que pode ser apresentado como o **Cenário 3**, mas antes de falar sobre uv, vamos dar um panorama geral do **pyproject**.

O Surgimento do pyproject.toml e a Base dos Gerenciadores Modernos

Durante muitos anos, o empacotamento Python foi baseado em um modelo fragmentado e fortemente dependente do setuptools. Projetos eram definidos principalmente por meio do arquivo setup.py, frequentemente acompanhado de setup.cfg, MANIFEST.in, requirements.txt e outros arquivos auxiliares. O setup.py não era apenas um arquivo de configuração: era um script Python executável que definia como o pacote deveria ser construído e instalado.

Quando alguém executava **pip install .**, o pip simplesmente rodava esse script. Ou seja, o processo era imperativo: o instalador executava código Python diretamente, sem uma interface formal e padronizada entre quem instala e quem constrói o pacote. Não existia uma API intermediária clara. Na prática, isso criava um acoplamento estrutural entre o ecossistema e o setuptools, pois o pip dependia diretamente desse modelo para funcionar. Criar alternativas era possível, mas exigia reproduzir o mesmo comportamento esperado pelo pip.

Esse modelo não afetava apenas quem desenvolvia bibliotecas públicas. Mesmo para quem mantinha aplicações internas, sistemas corporativos ou pipelines de dados, os problemas apareciam quando o projeto crescia. Não havia uma separação clara entre:

- metadados do projeto (nome, versão, compatibilidade de Python),
- dependências necessárias para execução,
- dependências necessárias apenas para build,
- arquivos que deveriam entrar na distribuição,
- lógica de construção do pacote.

Além disso, o fluxo completo exigia combinar manualmente múltiplas ferramentas: ambientes virtuais para isolamento, pip para instalação, setuptools para build, twine para publicação, e muitas vezes pip-tools para gerar dependências reprodutíveis. Não existia um ponto único de verdade para o projeto. Parte da configuração ficava em setup.py, parte em requirements.txt, parte em arquivos auxiliares. Cada ferramenta adotava convenções próprias.

Mesmo para quem nunca publicou nada no PyPI, isso se tornava um problema quando era necessário:

- garantir reprodutibilidade entre máquinas,
- versionar corretamente uma aplicação,
- automatizar builds em CI/CD,

- gerar artefatos instaláveis para produção,
- organizar um projeto que deixou de ser apenas um script isolado.

A situação começou a mudar com a introdução de três PEPs fundamentais. A **PEP 518** criou o arquivo **pyproject.toml**, permitindo declarar explicitamente qual sistema de build um projeto utiliza e quais dependências são necessárias para construí-lo. A **PEP 517** definiu uma interface padronizada entre instaladores (como o pip) e sistemas de build, eliminando a execução direta do setup.py. Posteriormente, a **PEP 621** padronizou os metadados do projeto dentro da seção [project], consolidando o pyproject.toml como o centro declarativo da configuração.

Com isso, projetos Python deixaram de depender de scripts executáveis para definir seu build e passaram a utilizar um modelo declarativo e padronizado. O **pyproject.toml** tornou-se o ponto único de configuração, separando claramente metadados, sistema de build e configurações específicas de ferramentas.

O pyproject.toml passou então a desempenhar três papéis centrais. Primeiro, declarar o sistema de build por meio da seção [build-system], permitindo que ferramentas como pip, build ou uv construam o projeto sem depender de uma implementação específica. Segundo, centralizar os metadados do projeto: nome, versão, descrição, dependências e scripts de forma padronizada. Terceiro, servir como espaço extensível para configurações específicas de ferramentas, por meio da convenção [tool.<nome_da_ferramenta>].

O formato TOML (Tom's Obvious Minimal Language) foi escolhido por ser simples, legível e não ambíguo. Por ser um formato estritamente declarativo, o arquivo pode ser analisado por qualquer ferramenta sem efeitos colaterais. Um pyproject.toml moderno e bem estruturado costuma ter três blocos principais.

O primeiro é **[build-system].** Essa seção é obrigatória para qualquer projeto distribuível. Ela define qual backend será usado para construir o pacote e quais dependências são necessárias para isso. Por exemplo:

```
None
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Aqui, o projeto declara que utiliza o hatchling como backend. O pip, ao instalar o projeto, consulta essa seção e chama o backend apropriado por meio da API padronizada da PEP 517. Essa separação entre instalador e sistema de build é o que removeu a dependência direta do setup.py.

O segundo bloco é **[project]**, definido pela PEP 621. Essa seção concentra os metadados do projeto de forma padronizada e interoperável. Um exemplo mínimo seria:

```
None
[project]
name = "meu-projeto"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = [
    "requests (>=2.32.4,<3.0.0)",
    "pandas (>=2.3.1,<3.0.0)",
]
```

Aqui ficam declarados:

- nome oficial do projeto
- versão
- versão mínima de Python
- dependências necessárias para execução

Observação: As dependências definem uma versão mínima aceitável e permitem atualizações que não introduzem mudanças incompatíveis segundo o versionamento semântico.

Opcionalmente, também podem ser incluídos:

- descrição
- licença
- autores
- classificadores
- URLs
- scripts de entrada

Essa seção representa a identidade formal do projeto. É o equivalente moderno do que antes ficava espalhado entre setup.py, setup.cfg e outros arquivos.

O terceiro bloco é **[tool]**. Ele não faz parte do padrão de empacotamento em si, mas é um espaço reservado para configurações específicas de ferramentas. Por exemplo:

```
None
      [tool.ruff]
      line-length = 100
```

Ou, no caso do Poetry:

```
None
      [tool.poetry]
      ...
```

Essa organização transforma o pyproject.toml no ponto único de configuração do projeto, centralizando não apenas o empacotamento, mas também linter, type checker, formatter e outras ferramentas de desenvolvimento.

Como um pyproject.toml deveria ser estruturado

Para um projeto moderno, seja uma biblioteca, uma aplicação interna ou um pipeline que precisa ser versionado e reproduzível, o arquivo ideal deve seguir alguns princípios:

1. Sempre declarar explicitamente o build-system.
2. Usar a seção [project] para metadados sempre que possível (evitando metadados proprietários quando o padrão já cobre o caso).
3. Declarar claramente requires-python para evitar instalações incompatíveis.
4. Separar dependências obrigatórias de dependências opcionais (extras).
5. Evitar duplicar informações em requirements.txt quando o projeto já está formalmente definido.

Um exemplo mais completo e alinhado com boas práticas seria:

```
None
      [project]
      name = "meu-projeto"
      version = "0.1.0"
      description = "Projeto interno de processamento de dados"
      authors = [
            {name = ,email = }
      ]

      requires-python = ">=3.10"
```

```
dependencies = [
   "pandas (>=2.3.1,<3.0.0)",
   "requests (>=2.32.4,<3.0.0)"
]

[project.optional-dependencies]
dev = ["pytest", "ruff"]

[project.scripts]
meu-comando = "meu_projeto.main:run"


[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

Esse modelo é:

- Declarativo
- Interoperável
- Compatível com pip, uv, build e outras ferramentas
- Independente de um gerenciador específico

Desta forma, o pyproject.toml resolve dois problemas fundamentais:

- Primeiro, remove a execução de código arbitrário durante o build, tornando o processo mais previsível e seguro.
- Segundo, estabelece um contrato formal entre projeto e ferramentas. Isso permite que diferentes gerenciadores de projeto (Poetry, Hatch, PDM, uv) operem sobre a mesma base estrutural.

Ele não é um gerenciador de dependências por si só. Ele é o padrão que permite que gerenciadores existam.

Embora o pyproject.toml tenha resolvido o problema estrutural da padronização e do desacoplamento entre instaladores e sistemas de build, ele não define como o desenvolvedor deve trabalhar no dia a dia. **Ele especifica a estrutura do projeto, mas não oferece, por si só, um fluxo integrado para gerenciar ambientes, dependências, resolução de versões e publicação.**

É nesse ponto que surgem os **gerenciadores de projeto modernos**. Eles não substituem o pyproject.toml; ao contrário, utilizam-no como base declarativa

padronizada. O que muda é o nível de abstração que oferecem. Ferramentas como o **Poetry,** criam e controlam o pyproject.toml, estruturando o projeto ao seu redor e oferecendo um fluxo completo que integra ambiente virtual, resolução de dependências, geração de lockfile, build e publicação. Já ferramentas como o **uv,** que será abordado logo mais**,** também utilizam o pyproject.toml, mas de maneira mais leve: podem ler e atualizar suas dependências, gerar um lockfile próprio e sincronizar o ambiente, sem impor um modelo rígido de organização do projeto.

Entre essas ferramentas, o Poetry foi uma das primeiras a consolidar essa experiência integrada, transformando o pyproject.toml no centro operacional do desenvolvimento. O uv, por sua vez, prioriza desempenho e simplicidade, oferecendo um mecanismo extremamente rápido para criar ambientes e resolver dependências, utilizando o mesmo padrão como fonte de configuração.

uv, um substituto moderno, rápido e unificado para pip e virtualenv

Para maiores informações, acesse a documentação oficial do uv:

Installation | uv
Features | uv

Dando um passo além do que foi discutido sobre **venv**, **virtualenv** e **pyproject.toml**, o **uv** representa uma evolução significativa na forma como ambientes e dependências são gerenciados no ecossistema Python. **Ele não se limita a substituir pontualmente ferramentas como venv ou pip; na prática, propõe uma consolidação de várias responsabilidades tradicionalmente distribuídas entre múltiplas ferramentas**. O uv combina, em uma única interface, funcionalidades de criação de ambientes virtuais, instalação e resolução de dependências, gerenciamento de versões do Python e execução de aplicações, mantendo compatibilidade com padrões estabelecidos como **pyproject.toml.**

Enquanto o fluxo tradicional de ambientes virtuais no Python costuma separar responsabilidades em dois passos, primeiro criar o ambiente com python -m venv (ou virtualenv) e depois gerenciar pacotes com pip, o uv unifica essas etapas em uma única ferramenta coerente. **Na prática, ele assume o papel do virtualenv como criador de ambientes e do pip como instalador e resolvedor de dependências, mantendo compatibilidade prática com ambos, mas oferecendo ganhos substanciais em velocidade, previsibilidade e confiabilidade.**

Uma diferença fundamental é que o uv foi escrito em Rust, e não em Python. Essa escolha técnica tem consequências diretas e mensuráveis: resolução de dependências, instalação de pacotes e criação de ambientes são significativamente mais rápidas e consomem menos recursos. Em contextos de

engenharia de dados e MLOps, onde pipelines de CI/CD frequentemente precisam criar ambientes do zero, instalar grandes conjuntos de dependências (como pandas, polars, pyspark, boto3, dbt, requests, entre outros) e repetir esse processo em múltiplas máquinas ou contêineres, essa diferença de desempenho se traduz em economia real de tempo e custo computacional.

Além da velocidade, o uv introduz melhores padrões por design. Ele parte do princípio de que trabalhar fora de um ambiente virtual é um antipadrão moderno. Diferentemente do pip tradicional, que permite (e até incentiva, por omissão) instalações no ambiente global, o uv assume que o usuário deve estar dentro de um ambiente isolado. **Se não houver um ambiente ativo ou um diretório .venv no projeto, o uv se recusa a instalar pacotes, evitando um dos erros mais comuns e perigosos no ecossistema Python: a contaminação do interpretador global do sistema.**

Quando se cria um ambiente com:

uv venv

O uv gera, por padrão, um diretório .venv no projeto, alinhando-se às convenções modernas já adotadas por ferramentas como Poetry e por muitas organizações de engenharia. **Esse ambiente não é baseado no módulo padrão venv do Python, mas sim em uma implementação compatível com virtualenv, que oferece descoberta mais robusta de interpretadores e caching agressivo para tornar a criação de ambientes extremamente rápida e confiável, mesmo em máquinas com múltiplas versões de Python instaladas.**

A seleção do interpretador também é mais inteligente e explícita. Com:

uv venv --python 3.12

O usuário pode definir explicitamente a versão desejada naquela ambiente virtual, e o uv é capaz de localizar esse interpretador no sistema de maneira inteligente, examinando o PATH e, no Windows, integrando-se ao mecanismo do py --list-paths. Isso reduz fricções comuns em equipes onde diferentes desenvolvedores possuem configurações locais distintas.

Para verificar as versões disponíveis do Python, utilize:

uv python list

Este comando mostra todas as versões que o uv pode usar, incluindo:

1. versões já instaladas no sistema
2. versões instaladas pelo próprio uv
3. versões disponíveis para download

Para instalar uma versão específica, utilize:

uv python install 3.11

Para fixar(travar) a versão do Python que um projeto deve usar:

uv python pin 3.12

O comando uv python pin serve para fixar (travar) a versão do Python que um projeto deve usar. **Ele faz isso criando um arquivo chamado .python-version na raiz do projeto.** Esse arquivo funciona como uma instrução explícita para o uv: sempre que algum comando for executado dentro daquele diretório, como uv venv, uv run ou uv pip install, o uv deve utilizar exatamente aquela versão do Python especificada.

A partir desse momento, o diretório do projeto passa a declarar formalmente que a versão de Python esperada é 3.12, por exemplo. Quando você criar um ambiente virtual com uv venv, o uv primeiro lê esse arquivo, verifica se o Python 3.12 já existe na máquina e, caso não exista, ele próprio faz o download e instala essa versão. Em seguida, cria o ambiente virtual usando exatamente esse interpretador.

Isso resolve um problema muito comum em projetos de software: diferenças de versão do Python entre ambientes. Quando o projeto possui o arquivo .python-version, todos os comandos do uv passam a respeitar essa definição automaticamente. Assim, qualquer pessoa que clone o repositório e execute uv venv no projeto terá um ambiente criado com a mesma versão do Python. Isso melhora muito a reprodutibilidade do ambiente, algo especialmente importante em projetos de engenharia de dados, pipelines, APIs ou aplicações que dependem de versões específicas de bibliotecas.

Na prática, o fluxo comum de configuração de um projeto com uv costuma ser: primeiro escolher a versão do Python que o projeto deve usar (uv python pin 3.12), depois criar o ambiente virtual (uv venv). A estrutura final do projeto geralmente fica com arquivos como pyproject.toml, .python-version e o diretório .venv. A partir daí, qualquer comando uv executado dentro desse diretório seguirá automaticamente a versão fixada do Python.

Como o uv usa o pyproject.toml ?

uv pode operar em dois modos distintos:

1. Modo simples (substituto de pip)

uv pip install pandas

Nesse caso:

- Não é necessário pyproject.toml
- Não há lockfile
- Ele funciona como um pip acelerado

Esse modo é útil para scripts isolados ou ambientes experimentais.

2. Modo projeto (modelo moderno)

Se você executar:

uv init

O uv cria automaticamente um **pyproject.toml** e prepara o projeto para uso declarativo.

Ao adicionar dependências:

uv add pandas polars boto3

Ele:

- Atualiza o pyproject.toml
- Resolve a árvore completa de dependências
- Gera um uv.lock
- Instala no ambiente

Aqui, o pyproject.toml funciona como o lugar onde o projeto declara quais dependências ele precisa. Nesse arquivo normalmente indicamos apenas as restrições de versão, e não necessariamente a versão exata que será instalada. Por exemplo “pandas (>=2.3.1,<3.0.0)". Isso significa apenas que o projeto precisa de uma versão do pandas **a partir da 2.0 e anterior a 3.0**, mas não define exatamente qual versão será usada.

Quando o uv instala as dependências, ele analisa todas as bibliotecas necessárias e também as dependências internas de cada uma delas. A partir disso, ele calcula uma combinação de versões que funcione corretamente para todo o conjunto de pacotes. Depois que essa combinação é encontrada, o uv registra o resultado no arquivo **uv.lock**. Esse arquivo contém **as versões exatas de cada pacote que foram instaladas** no ambiente.

O objetivo do lockfile é garantir que qualquer pessoa consiga recriar exatamente o mesmo ambiente, instalando as mesmas versões de todas as dependências.

De forma simplificada:

- **o pyproject.toml descreve o que o projeto precisa**
- **o uv.lock registra exatamente o que foi instalado**

Ou seja, o pyproject.toml define as dependências desejadas, e o uv.lock guarda a solução final encontrada pelo resolvedor de dependências.

O uv também consegue ler pyproject.toml de um projeto já iniciado.

Há também uma dimensão estratégica no design do uv. Seu objetivo declarado é evoluir para uma ferramenta unificada de empacotamento para Python, semelhante ao papel que o Cargo desempenha no ecossistema Rust. Isso significa menos fragmentação entre ferramentas (pip, virtualenv, pipenv, poetry, etc.) e mais coerência no fluxo de trabalho. Mesmo em estágio inicial, o uv já oferece um conjunto de funcionalidades integradas, com padrões sensatos e uma experiência mais consistente para desenvolvedores.

Em comparação direta com o fluxo tradicional de venv + pip, o uv oferece:

- Desempenho superior, por ser implementado em Rust e otimizado para resolução rápida de dependências.
- Melhor segurança operacional, ao evitar instalações acidentais no ambiente global.
- Experiência mais moderna, alinhada com práticas atuais de desenvolvimento e DevOps.
- Menos fricção em CI/CD, onde velocidade e confiabilidade são críticas.
- Descoberta de Python mais robusta, facilitando o trabalho em máquinas com múltiplas versões instaladas.

Comparação com Poetry

Comparado a gerenciadores de projeto como o **Poetry**, o **uv** tende a ser mais flexível na camada de execução e gerenciamento de ambientes. Enquanto o Poetry oferece um fluxo altamente integrado, que inclui estruturação do projeto, gerenciamento de dependências, build e publicação, o uv concentra-se principalmente na criação de ambientes e na resolução de dependências, priorizando desempenho e simplicidade operacional.

Isso significa que o uv não impõe um modelo rígido de organização do projeto nem um fluxo fechado de empacotamento. Em vez disso, ele fornece uma base extremamente rápida e confiável para criação de ambientes virtuais e instalação de dependências, mantendo alta compatibilidade com o ecossistema Python existente.

Por outro lado, o **Poetry continua sendo extremamente útil quando o objetivo principal é estruturar e publicar um pacote Python formalmente**, como uma biblioteca interna, um SDK corporativo ou um projeto que será distribuído via PyPI. Nesse tipo de cenário, o modelo opinativo do Poetry, com gerenciamento centralizado de metadados, build e publicação, pode ser vantajoso.

Para profissionais que trabalham com **pipelines complexos, múltiplos projetos e ambientes distribuídos**, o uv não é apenas uma ferramenta mais rápida. Ele pode servir como uma fundação mais enxuta e previsível para construir ambientes reprodutíveis e escaláveis ao longo do tempo.

Migrando do Poetry para uv

Para equipes que utilizam Poetry há algum tempo, a adoção do uv pode trazer algumas vantagens operacionais importantes.

Uma das primeiras diferenças percebidas é que o uv cria automaticamente o ambiente virtual dentro do próprio projeto, geralmente no diretório .venv. Isso evita o uso de diretórios internos do sistema, como ocorre em algumas configurações do Poetry e torna mais fácil localizar e inspecionar o ambiente utilizado pelo projeto.

Outra vantagem significativa é o **desempenho**. A resolução da árvore de dependências no uv costuma ser muito mais rápida, especialmente em projetos que utilizam bibliotecas pesadas ou com cadeias complexas de dependências.

O uv também pode **instalar automaticamente a versão necessária do Python** caso ela não esteja disponível no sistema, reduzindo atritos na configuração de novos ambientes de desenvolvimento.

Além disso, ele oferece maior flexibilidade para trabalhar com **múltiplas versões de Python**, facilitando cenários em que diferentes projetos exigem versões distintas do interpretador.

Em ambientes de **CI/CD e containers**, essas características se traduzem em builds mais rápidos, ambientes mais previsíveis e menos falhas relacionadas à resolução de dependências

Poetry

A introdução do **pyproject.toml** trouxe uma mudança importante para o ecossistema Python. Ao padronizar os metadados dos projetos e a forma como sistemas de build são declarados, ele quebrou o monopólio histórico do setuptools.

No entanto, mesmo com essa padronização, alguns desafios permanecem. **Trabalhar com um projeto baseado em pyproject.toml ainda exigia entender diversos componentes do ecossistema Python: fluxos de empacotamento, diferentes ferramentas de build e várias opções para gerenciamento de dependências**. Além disso, o desenvolvedor precisa escolher entre diferentes backends de build e muitas vezes nem está claro o que esses backends são ou quando usar cada um.

Em muitos fluxos tradicionais, é necessário declarar dependências manualmente e depois gerar arquivos adicionais, como requirements.txt, usando ferramentas como pip-tools. Isso acaba poluindo o projeto com múltiplos arquivos de configuração. Além disso, em um sistema de desenvolvimento típico, é comum acumular diversos ambientes Python diferentes, o que pode tornar difícil acompanhar qual ambiente pertence a qual projeto.

Foi nesse contexto que surgiu o **Poetry.**

O Poetry é um gerenciador de projetos Python criado para simplificar o fluxo de desenvolvimento. Ele foi desenvolvido inicialmente por Sébastien Eustace em 2018 e rapidamente ganhou popularidade por oferecer uma abordagem integrada para três tarefas que tradicionalmente eram feitas com ferramentas separadas:

- gerenciamento de dependências
- criação de ambientes virtuais
- empacotamento e publicação de projetos

Em vez de exigir várias ferramentas distintas, o Poetry oferece um conjunto único de comandos que cobre todo o ciclo de desenvolvimento de um projeto Python.

Internamente, o Poetry também possui seu próprio backend de build, chamado poetry-core. Esse backend segue os padrões do ecossistema Python, o que significa que projetos criados com Poetry continuam compatíveis com ferramentas padrão como pip, build e uv.

Outro recurso importante do Poetry é o seu resolvedor de dependências. Ele calcula automaticamente uma combinação de versões compatíveis entre todas as bibliotecas do projeto e registra o resultado em um arquivo chamado **poetry.lock.** Esse arquivo garante que todos os desenvolvedores do projeto e todos os ambientes de deployment utilizem exatamente as mesmas versões das dependências.

Mais recentemente, ferramentas como o uv, desenvolvido pela Astral, também começaram a ganhar destaque no ecossistema.

Instalando o Poetry

Para maiores informações, acesse a documentação oficial do Poetry:

[Introduction | Documentation | Poetry - Python dependency management and packaging made easy]

A forma recomendada de instalar o Poetry é utilizando o **pipx**, **que instala ferramentas Python globais em ambientes isolados.**

pipx install poetry

Isso garante que as dependências do Poetry não interfiram com outras bibliotecas instaladas no sistema. Uma única instalação do Poetry pode funcionar com múltiplas versões de Python. No entanto, ele utiliza o interpretador Python usado durante sua instalação como versão padrão. Por isso, geralmente é recomendado instalar o Poetry utilizando a versão mais recente do Python disponível.

Se uma nova versão do Python for instalada no sistema, é possível reinstalar o Poetry utilizando:

pipx reinstall --python=3.12 poetry

Também existe um instalador oficial do Poetry que pode ser executado diretamente com Python:

curl -sSL https://install.python-poetry.org | python3 -

Esse método é menos flexível do que o uso de pipx, mas pode ser conveniente em alguns cenários. Após a instalação, você pode verificar se o Poetry está funcionando executando simplesmente:

poetry --version

O comando exibirá a versão instalada.

Metadados do projeto no Poetry

Ao criar um projeto com Poetry, o arquivo pyproject.toml já vem com uma estrutura básica de metadados.

Por exemplo:

```
None
[project]
name = "random-wikipedia-article"
version = "0.1.0"
description = ""
authors = ["Your Name <you@example.com>"]
readme = "README.md"
```

Gerenciando dependências com Poetry

Uma das principais funcionalidades do Poetry é simplificar o gerenciamento de dependências. Em vez de editar manualmente arquivos de configuração, você pode adicionar bibliotecas diretamente pela linha de comando.

Por exemplo:

poetry add rich

Ao executar esse comando, o Poetry realiza várias ações automaticamente:

1. adiciona a dependência no pyproject.toml
2. resolve a árvore completa de dependências
3. instala os pacotes no ambiente virtual do projeto
4. atualiza o arquivo poetry.lock

O arquivo poetry.lock

Assim como outras ferramentas modernas de gerenciamento de dependências, o Poetry utiliza um arquivo de lock para registrar exatamente quais versões de pacotes foram instaladas.

Esse arquivo se chama:

poetry.lock

Ele contém:

- todas as dependências diretas
- todas as dependências transitivas
- versões exatas de cada pacote
- hashes criptográficos dos arquivos distribuídos

O objetivo do poetry.lock é garantir **reprodutibilidade**. Isso significa que qualquer pessoa que instalar o projeto, ou qualquer pipeline de CI/CD receberá exatamente o mesmo conjunto de dependências.

Para visualizar dependências instaladas, é possível usar:

poetry show

Ou visualizar a árvore completa:

poetry show --tree

Se o pyproject.toml for modificado manualmente, o lockfile pode ser atualizado com:

poetry lock --no-update

Atualizando dependências

O Poetry permite atualizar dependências de maneira controlada. Para atualizar todas as dependências:

poetry update

Para atualizar apenas um pacote específico:

poetry update rich

Esse comando atualiza as versões dentro dos limites definidos no pyproject.toml. Para remover uma dependência:

poetry remove pacote

Gerenciamento de ambientes virtuais

O Poetry também cuida automaticamente da criação e manutenção de ambientes virtuais. Quando você adiciona dependências ou instala o projeto, o Poetry cria um ambiente virtual se necessário.Por padrão, esses ambientes são armazenados em um diretório global gerenciado pelo Poetry. No entanto, é possível configurá-los para ficarem dentro do próprio projeto:

poetry config virtualenvs.in-project true

Isso cria um ambiente no diretório .venv, o que facilita a integração com outras ferramentas. Para verificar o ambiente atual:

poetry env info --path

Para recriar o ambiente com uma versão específica do Python:

poetry env remove --all

poetry env use 3.12

Instalando o projeto

Depois de criar um projeto com Poetry ou clonar um repositório que utiliza essa ferramenta, o próximo passo é instalar o projeto no ambiente virtual. Isso é feito utilizando o comando:

poetry install

Esse comando é responsável por preparar completamente o ambiente de execução do projeto. Quando executado, o Poetry realiza automaticamente uma série de etapas:

1. Primeiro, ele verifica se já existe um ambiente virtual associado ao projeto. Caso não exista, o Poetry cria um novo ambiente virtual para garantir o isolamento das dependências.
2. Em seguida, o Poetry lê o arquivo pyproject.toml, que contém a declaração das dependências do projeto, e o arquivo poetry.lock, que registra as versões exatas das bibliotecas que devem ser instaladas.
3. Com base nesses arquivos, o Poetry instala todas as dependências necessárias no ambiente virtual. Por fim, ele também instala o próprio projeto no ambiente, geralmente utilizando uma **instalação editável**. Isso significa que qualquer alteração feita no código-fonte do projeto passa a refletir imediatamente no ambiente, sem a necessidade de reinstalar o pacote.

Na prática, o comando poetry install prepara tudo que é necessário para começar a trabalhar no projeto. Depois da instalação, existem duas formas comuns de executar código dentro do ambiente gerenciado pelo Poetry.

A primeira é abrir um shell dentro do ambiente virtual:

poetry shell

Esse comando ativa o ambiente virtual e inicia uma sessão de terminal dentro dele. A partir desse ponto, qualquer comando Python executado utilizará o ambiente do projeto. Outra alternativa é executar comandos diretamente dentro do ambiente sem abrir um shell interativo. Isso pode ser feito utilizando:

poetry run python

ou

poetry run <comando>

Por exemplo:

poetry run python script.py

Nesse caso, o Poetry executa o comando dentro do ambiente virtual do projeto, sem precisar ativá-lo manualmente. Essa abordagem torna o fluxo de trabalho mais simples, pois o Poetry passa a cuidar automaticamente da criação do ambiente, da instalação das dependências e da execução do projeto, reduzindo a necessidade de gerenciar manualmente ambientes virtuais e bibliotecas instaladas.

pipx: execução segura de ferramentas isoladas

O pipx foi criado para resolver um problema prático comum em equipes de desenvolvimento, mas que também pode ser um problema para área de dados: a necessidade de instalar e usar ferramentas de linha de comando em Python (como dbt, black, ruff, awscli, jupyter, datasette, entre outras) em vários projetos, sem misturar essas ferramentas com as dependências específicas de cada projeto e sem modificar o Python global do sistema.

Em vez de instalar essas ferramentas no Python do sistema, o pipx segue um princípio simples e poderoso: **cada ferramenta é instalada em seu próprio ambiente virtual dedicado**. Do ponto de vista do usuário, ela parece instalada "globalmente", mas internamente está isolada.

Ou seja, a diferença fundamental do pipx em relação ao uso tradicional de ambientes virtuais é a **unidade de isolamento**. Enquanto um ambiente virtual comum isola **todas as dependências de um projeto dentro de um único ambiente**, o pipx isola **cada ferramenta em seu próprio ambiente virtual dedicado**, mas a torna acessível como se estivesse instalada "globalmente".

Na prática, isso significa que:

- Um ambiente virtual tradicional (criado com venv ou uv venv) é usado para executar o código de um projeto específico e contém as bibliotecas desse projeto.
- O pipx é usado para instalar ferramentas que você utiliza em vários projetos, independentemente das dependências internas de cada um.

Por exemplo, ao executar:

pipx install dbt-core

O pipx:

1. Cria automaticamente um ambiente virtual exclusivo para o dbt-core.

2. Instala o pacote neste ambiente isolado.
3. Disponibiliza o comando dbt no seu PATH, de modo que você possa usá-lo em qualquer projeto.

Se depois você instalar outra ferramenta:

pipx install black

Ela ficará em outro ambiente virtual separado, evitando conflitos de versões entre ferramentas.

Isso é diferente de instalar diretamente no Python global do sistema. Instalar ferramentas globalmente com pip install pode:

- interferir no Python gerenciado pelo sistema operacional,
- gerar conflitos entre dependências de ferramentas distintas,
    - Se duas ferramentas exigirem versões incompatíveis de uma dependência, você simplesmente não consegue instalar as duas.
- dificultar a manutenção e a reprodutibilidade do ambiente de trabalho.

Com pipx, você obtém o melhor dos dois mundos:

- ferramentas **compartilhadas entre projetos**, mas **isoladas tecnicamente em seus próprios ambientes virtuais.**

Essa abordagem traz vantagens importantes para profissionais de dados:

- **Isolamento real entre ferramentas**, reduzindo conflitos de dependências;
- **Estabilidade do ambiente de trabalho**, pois atualizar uma ferramenta não quebra outras;
- **Ambientes de projeto mais limpos**, contendo apenas dependências do código e não utilitários de desenvolvimento;
- **Portabilidade pessoal**, permitindo manter o mesmo conjunto de ferramentas em diferentes máquinas.

Também é possível executar uma ferramenta sem instalá-la permanentemente:

pipx run black .

Isso é útil em automações, testes rápidos ou ambientes compartilhados.